UNIDAD 4: DESARROLLO JUEGOS 2D y 3D

Módulo Profesional:
Programación Multimedia y Dispositivos Móviles

Índice

RESUMEN INTRODUCTORIO 3
INTRODUCCIÓN 3
CASO INTRODUCTORIO 3
1. DEFINICIÓN DE JUEGO 4
1.1 Pequeña historia del videojuego 5
1.2 Diseño de videojuegos 7
2. MOTORES DE VIDEOJUEGOS 10
2.1 Ejemplos y tipos 11
2.2 Características esenciales 12
3. MOTOR DE VIDEOJUEGOS 2D 13
3.1 Instalación 13
3.2 Presentación del proyecto 14
3.3 Comenzando por el fondo del proyecto 14
3.4 Incorporar un jugador 15
3.5 Incorporar comportamientos 16
3.6 Incorporar eventos 16
3.7 Incorporar naves enemigas 17
3.8 Crear jugabilidad en el juego 18
3.9 Finalizando el juego 18
4. MOTOR DE VIDEOJUEGOS 3D 20
4.1 Instalación 21
4.2 Interfaz de Unity 21
4.3 Añadiendo objetos a la escena 24
4.4 Transformando objetos 26
4.5 Iluminación 29
4.6 Materiales y texturas 31
4.7 Scripts, lenguajes y API gráfico 36
RESUMEN FINAL 40

RESUMEN INTRODUCTORIO

En esta unidad nos introduciremos en el desarrollo de videojuegos y la utilización de dos motores de videojuegos tanto 2D como 3D. En la actualidad la diversidad de motores de videojuegos es muy grande y la competencia entre ellos también, lo que es siempre muy beneficioso desde nuestro punto de vista como desarrolladores.

También introduciremos las técnicas básicas del diseño de videojuegos, tan importantes cuando se va a construir un videojuego.

INTRODUCCIÓN

El mercado del videojuego tiene una historia ridícula si lo comparamos con la historia de otros ámbitos profesionales y, sin embargo, su crecimiento ha sido espectacular siendo en estos momentos uno de los sectores con más proyección y recursos.

Dentro de este mercado al que todos estamos acostumbrados, tienen cabida profesionales del mundo artístico y, por supuesto, del desarrollo software que es el que nos interesa. El desarrollo de videojuegos también ha evolucionado enormemente, acercándose cada vez más al usuario final, haciéndonos la vida cada vez más sencilla, y poniendo a nuestro alcance software denominados motores de videojuegos más potentes y con más posibilidades.

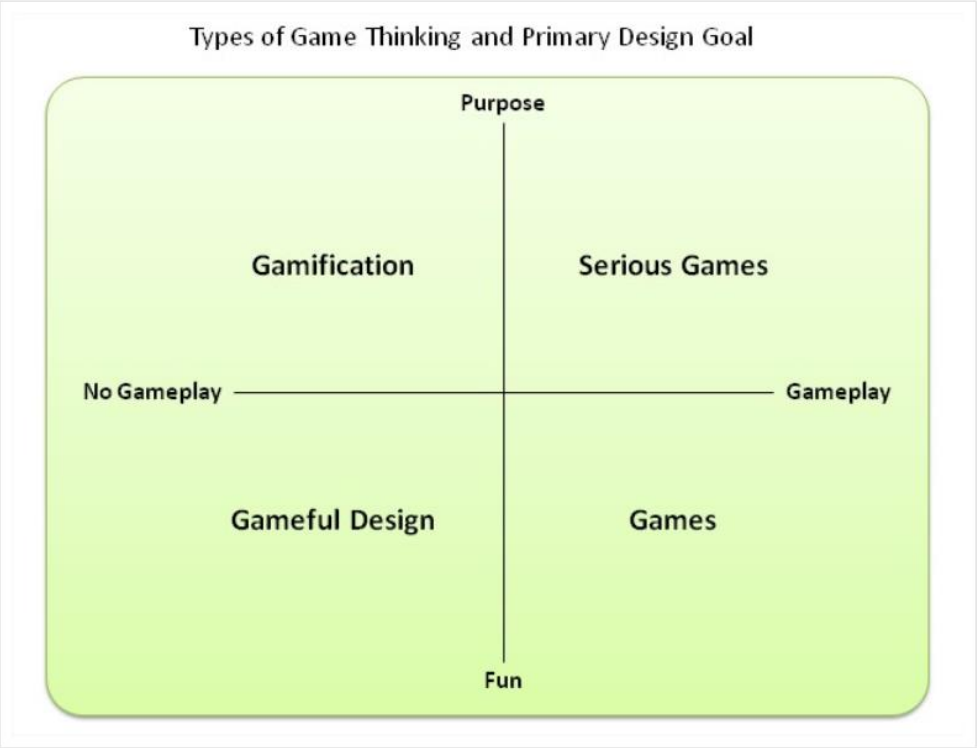
Como veremos a lo largo de esta unidad, nos encontramos en un momento dorado donde tenemos múltiples opciones de motores, 2D, 3D, multiplataforma, especializados, gratuitos, de pago, y que, como característica común, nos permitirán realizar todas las fases del diseño de un videojuego.

CASO INTRODUCTORIO

Queremos realizar un videojuego 2D o 3D que una escuela de secundaria necesita para poder motivar a su alumnado en el estudio de la historia.

1. DEFINICIÓN DE JUEGO

Para comprender los videojuegos y su estado actual, es muy importante tener una pequeña perspectiva de su historia y de los juegos en general. ¿Qué es un juego? Según la RAE, acción y efecto de jugar por entretenimiento. Si nos fijamos en el siguiente cuadrante:



Cuadrante de clasificación de los juegos.
Fuente: http://www.hisocial.com/esp/wp-content/uploads/2013/09/gamification-serious-games.png

Los juegos se encuentran en el cuadrante justamente orientados al entretenimiento, a la acción de jugar, pero esto no quita que la creación de juegos y/o videojuegos estén relacionados con actividades orientadas al mundo profesional como pueda ser el entrenamiento médico o los simuladores de vuelo. ¿Qué nos atrae de ellos? Nos obligan a tomar decisiones, competir y trabajar juntos. No es sorprendente que nuestro amor por los juegos haya entrado en los límites del mundo virtual, ya que siempre estamos tratando de hacer un juego de todo lo que hacemos.

1.1 Pequeña historia del videojuego

Podríamos decir que en abril de 1972 el mundo del videojuego experimentó el nacimiento de algo nuevo. Su nombre fue Magnavox Odyssey y su creador Ralph Baer, (Hadzinsky & College, 2014).

Magnavox Odyssey.
Fuente: https://upload.wikimedia.org/wikipedia/commons/thumb/0/0f/Magnavox-Odyssey-Console-Set.png/1200px-Magnavox-Odyssey-Console-Set.png

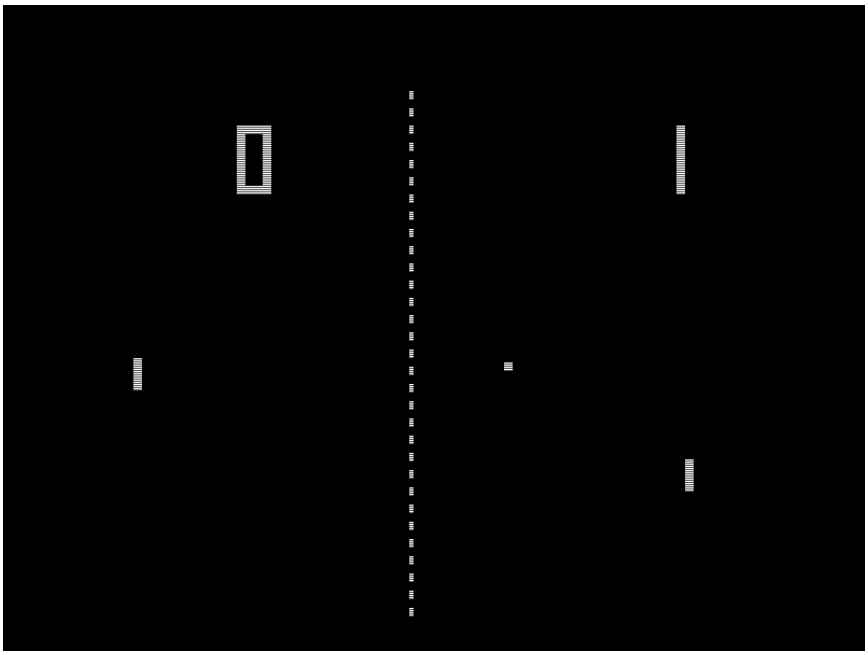
Como se observa en la imagen, fue una de las primeras o la primera videoconsola y que provocó la venta de 100000 unidades, el primer gran éxito de las consolas de consumo.

ENLACE DE INTERÉS

En el siguiente enlace encontrarás un timeline de la historia del videojuego:

http://www.icheg.org/icheg-game-history/timeline/

Si antes hablábamos del primer éxito en consolas, les toca el turno a los videojuegos. PONG fue la gran sensación lanzada por ATARI, y no solo con la dinámica de los éxitos de videojuegos, sino de algo que después llamaríamos "sagas" o "clones" debido a que no se tenía conciencia de incluir dentro de las patentes a los videojuegos y a los tres meses se encontraba en un problema legal con la cantidad de videojuegos idénticos a PONG.



PONG.
Fuente: https://upload.wikimedia.org/wikipedia/commons/f/f8/Pong.png

En la década de los 80, se produce un incremento en el mercado de los videojuegos, tanto que supera al mercado de la música y Hollywood, desarrollándose dos mercados, el mercado del arcade y el mercado doméstico. Es en este entorno donde terceras empresas comienzan a desarrollar juegos para consolas como Atari y donde también comienza a introducirse los ordenadores personales. Es en este cultivo, en 1989 donde tiene su primer gran éxito uno de los gigantes que ahora conocemos, NINTENDO, con la venta de 7 millones de consolas.

PARA SABER MÁS

En el siguiente enlace podrás aprender la historia de las consolas de Nintendo:

https://www.enter.co/cultura-digital/videojuegos/una-historia-visual-de-todas-las-consolas-de-nintendo/

Dos cambios tecnológicos en los 90 y a partir del siglo 21 producen también un cambio en la concepción de los videojuegos y un crecimiento de estos. En los años 90 nos encontramos con una continua mejora del hardware, se incrementan las velocidades de las CPUs, aparecen la GPU, se incrementa la capacidad de almacenamiento. En el siglo XXI, Internet lo marca todo. La distribución se realiza por Internet, pero el cambio más importante es el juego multijugador.

¿Dónde estamos ahora? Podríamos decir que el iPhone e iPad lo vuelve a cambiar todo. Se produce una revolución en el concepto de juego apareciendo lo que se denomina juego casual. Lo que grandes empresas tenían el monopolio (y lo siguen teniendo en ese entorno) con el juego AAA, ahora mismo, pequeños estudios, se permiten el lujo de realizar grandes proyectos y juegos, sobre todo, para los dispositivos móviles.

Es aquí, en este entorno del desarrollo para móviles y tablets, donde está el mercado de nuestros desarrollos, tanto en el entorno del juego puro como en el entorno de la gamificación y serius games.

1.2 Diseño de videojuegos

Como ocurría en las películas y televisión antes, los videojuegos están en una era dorada cuando el mercado está abierto a pequeños equipos e incluso a individuos para hacer su propio producto. Los títulos independientes nunca han sido tan buenos y hay una montaña de diferentes herramientas que pueden ayudarte.

La realidad es que cualquiera puede hacer un juego, el proceso puede ser tan simple o complejo como desee. El proceso de hacer un juego desde cero se puede dividir en solo cuatro pasos:

- Conceptualización: Decidir cómo va a ser el juego, su parte gráfica, cómo juega y de qué se trata.

- Arte: Creando sprites, texto, modelos e interfaz de usuario para tu juego.

- Programación: Escribir los sistemas que determinan cómo el jugador interactúa con el mundo del juego.

- Distribución y publicación.

Conceptualización

La diferencia entre un juego amateur y un juego profesional principalmente vendrá marcada por aquello que no se ve. La planificación adecuada es el paso más importante en cualquier proyecto y el diseño de los videojuegos no es una excepción.

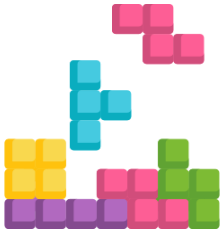
La fase de conceptualización del proyecto actúa como un mapa de ruta para que el equipo de trabajo lo siga. Dado que los videojuegos pueden ser

bastante amplios, con muchos sistemas y recursos, es fácil perderse. Un plan sólido ayuda a prevenir esto.

La primera pregunta que debemos hacernos es: "¿Qué puedo hacer?". Esto es sutilmente diferente de "¿Qué quiero hacer?", porque cualquier proyecto debe tener en cuenta factores como las habilidades, el tiempo y el dinero.

Por resumir este apartado, podríamos hacer la siguiente lista de necesidades para la conceptualización:

- **Ideas.**
- **Documento de diseño:** Notas escritas que sirven de guía para nuestro juego. Esto puede incluir todo, desde ideas de historias, descripciones de personajes e instrucciones para diseñadores. Cuanto más detallado sea ahora, menos tendremos que preocuparnos más tarde.
- **Arte conceptual:** El arte es muy importante en la preproducción, ya que actúa como una hoja de ruta para el siguiente paso. Si tienes personajes, necesitas bocetos de personajes. Si tienes una historia, necesitas story boards. Nuevamente, es mejor tener más detalles, ya que las ideas siempre se pueden combinar para enfocarse en los puntos importantes de su objetivo.
- Lista de verificación de activos.



EJEMPLO PRÁCTICO

Para la realización de la parte conceptual de un videojuego dentro de un equipo de trabajo, se puede utilizar perfectamente una herramienta colaborativa como es Google Docs.

Para ello:

1) Crearíamos en Drive una carpeta compartida.
2) Crearíamos un documento compartido que se denominase ideas. En este documento cada miembro compartiría sus ideas del proyecto.
3) Crearíamos otro documento que se denominase documento de diseño.
4) La carpeta serviría como almacén del arte propuesto.

COMPRUEBA LO QUE SABES

Una vez vista la parte inicial de concepto, y a raíz de la propuesta anterior. ¿Qué herramientas colaborativas utilizarías para la parte de arte además de Drive de Google?

Comenta tu respuesta en el foro.

Arte

Después del proceso relativamente rígido de conceptualización, llegamos al paso mucho más libre de diseño de arte. Cada fondo, botón, barra de salud y cada fotograma de cada animación de personaje necesitará una obra de arte por separado. Aquí hay mucho espacio para los cambios y la improvisación.

Qué necesitaremos:

- **Material conceptual:** Tener todo el trabajo duro de la fase de preproducción hará que este paso sea mucho más fácil.
- **Una estación de trabajo.**

Programación

Hay herramientas disponibles gratuitamente para creadores de juegos de cualquier nivel de habilidad o experiencia. Este tipo de herramientas incluye bibliotecas de utilidades prefabricadas para su uso en un lenguaje de programación, paquetes de software basados en GUI como Unity o UDK, y herramientas de animación. Incluso si no quiere crear algo desde cero, hay muchas opciones disponibles.

Es el paso más exigente desde el punto de vista técnico, pero casi no hay requisitos previos para el aprendizaje y los recursos fáciles de usar están en todas partes. Hay cientos de lenguajes de programación para elegir y la elección puede ser desalentadora, pero aquí hay algunas buenas sugerencias.

Distribución y publicación

En este paso del proceso, nuestro juego ya está hecho, probado y listo para funcionar, pero ¿hacia dónde va? El éxito aquí se puede definir como queramos, por lo que termina siendo un paso muy nebuloso, pero quizás el más importante. ¿De qué sirve un juego si nadie lo juega?

Es increíblemente importante que las personas sepan que nuestro juego existe. Todos los estudios de juegos en ciernes también deben ser parte de la firma de relaciones públicas y la agencia de publicidad. Hoy en día, es fácil apegarse al correo no deseado de las redes sociales y tácticas virales, pero a la larga la publicidad responsable dará sus frutos con el respeto de sus usuarios.

2. MOTORES DE VIDEOJUEGOS

Estamos hoy en día acostumbrados a títulos de videojuegos como Final Fantasy, BattleField, Candy Crash, Clash o Clans y otros muchos, puesto que el mercado actual es enorme.

ENLACE DE INTERÉS

En el siguiente enlace de la asociación española de videojuegos encontraréis múltiples estadísticas como ventas de videojuegos por meses y anuales y otra información de interés.

http://www.aevi.org.es/

Sin embargo, ¿cómo se realizan esos videojuegos? Complicada pregunta que no tiene una respuesta cerrada y con una explicación sencilla. Si se trata de una gran producción, AAA como se denominan informalmente, se necesita un equipo enorme de gente, al igual que una producción de cine. Pero para un videojuego para móvil, tablet o web, podemos realizarlo con un estudio de gente mucho más pequeño y reducido.

Y ¿dónde entramos nosotros como programadores? Entramos en la programación del videojuego y para programar necesitaremos un motor de videojuegos, que no es ni más ni menos que un kit de desarrollo que contiene código fuente, librerías y mecanismos que nos permite desarrollar y probar videojuegos de una forma rápida. Es algo así como el Android de los videojuegos. Estos motores contienen físicas, algoritmos y un framework que permite al programador centrarse en lo que verdaderamente importa y es diferencial que es en la dinámica del juego y en su programación.

2.1 Ejemplos y tipos

Vivimos un momento de alternativas y, aunque muchos de los motores de videojuegos son gratuitos, sus modelos de negocio son diferentes y son utilizados por las grandes compañías. Un ejemplo de algunos motores 3D:

- CryEngine (https://www.cryengine.com): Potente motor de videojuegos utilizado por sagas como Warface o Crysis.
- Frostbite: Creado por EA y usado por Battlefield y otros títulos.
- Unreal (https://www.unrealengine.com): Creado por Epic Games y es uno de los más usados y conocidos para la creación de juegos AAA como Warseries, Unreal Tournament y otros.
- IW Engine: Usado en los juegos de Call of duty.
- Unity 3D (https://unity3d.com/es): Puede ser el más popular de los anteriores por su calidad, potencial y, sobre todo, por ser un software gratuito.

En todos los ejemplos anteriores ponemos motores basados en una parte 3D de diseño, motores muy potentes para la realización de videojuegos AAA, pero tal y como hemos comentado en más de una ocasión el mercado del videojuego se divide entre la consola, el pc y el dispositivo móvil. En este último, el uso de motores 3D deja de ser tan necesario y nos encontramos con motores 2D muy potentes, con mucha potencialidad con el objetivo en ese dispositivo algunos ejemplos:

- GameMaker: Excelente motor 2D basado en las posibilidades HTML+JS+CSS para la creación de videojuegos multiplataforma.
- Construct2: Otro motor 2D basado en HTML+JS+CSS.

ENLACE DE INTERÉS

Otro enlace fantástico sobre las diferentes opciones de motores de videojuegos:

https://www.vidaextra.com/listas/si-quieres-hacer-tus-propios-juegos-estos-son-los-mejores-motores-que-vas-a-encontrar

2.2 Características esenciales

A continuación, se expondrán una serie de características relevantes en los motores de videojuegos.

Interfaz de desarrollo

Este es el componente central de cualquier motor de juego, 3D o 2D, que vincula todos los subsistemas que se analizarán a continuación en un paquete limpio y fácil de usar. Su trabajo es representar de forma abstracta el mundo del juego y los objetos que residen en él, lo que facilita a los desarrolladores razonar acerca de su juego. Es muy común utilizar una separación dentro del interfaz entre la parte gráfica y la de programación.

Renderizado

La mayoría de los procesadores utilizan algún tipo de API de dibujo abstracto (por ejemplo, Direct3D y/o OpenGL) para embeber los gráficos 3D o 2D en la pantalla. Por lo general, se implementan como una clase en la que se pueden pasar objetos y parámetros. Contiene un bucle que itera sobre todos los objetos en el mundo del juego y los dibuja en la pantalla.

Audio

El subsistema de audio reproduce música de fondo y efectos de sonido en el juego. Si te gustan las bibliotecas de código abierto, OpenAL-Soft o PortAudio son buenas opciones para ayudarte a escribir un sistema de audio multiplataforma. Para decodificar formatos de audio, se recomienda integrar Xiph Theora (.ogg para audio) o Xiph Vorbis (.ogv para video) en tu juego, ya que ambos son eficientes y opensource.

Inteligencia Artificial (AI) y Físicas

Cada motor de juego debe tener alguna forma de inteligencia artificial que controle a los NPC (Personajes sin Jugador) y sus puntos de generación, así como las físicas cuando los personajes interactúan entre sí y con el escenario. Esto generalmente se logra con una biblioteca de terceros o incluida con el motor.

Otros subsistemas

Dentro del motor nos encontramos con otros subsistemas como los subsistemas de entrada y salida, el de redes, el de datos, internacionalización,… Todos ellos importantes y que en casi todos los videojuegos están presentes de una u otra forma.

3. MOTOR DE VIDEOJUEGOS 2D

Comenzaremos nuestro primer proyecto y videojuego utilizando un motor de videojuegos 2D. Como hemos dicho, tenemos ahora mismo muchas alternativas y motores, aunque muchas de ellas se basan en las tecnologías HTML5+CSS+JS para exportar el proyecto y producir un videojuego multiplataforma.

En concreto utilizaremos un fantástico motor, Construct2 de Scirra, con el cual podemos crear videojuegos de unas características fantásticas y muy profesionales.

3.1 Instalación

Construct 2 es un motor totalmente gratuito que funciona sobre Windows, así que para poderlo ejecutar sobre otras plataformas deberemos tener una instalación de Windows mediante Parallels, por ejemplo, en sistemas Mac o en otra partición mediante Linux.

Además, es un motor freeware, lo que significa que la versión gratuita nos permitirá ciertas características limitadas frente a la versión de pago. Aun así, la versión gratuita nos permitirá adentrarnos en el mundo del desarrollo y también tiene suficiente potencia para poder realizar muy buenas demos.

PARA SABER MÁS

Durante la escritura de este manual, aparece la nueva versión de Construct, Construct 3, estando en estos momentos ambas versiones disponibles.

EDITOR TOTALMENTE EN LA NUBE

https://www.construct.net/es/make-games/free-trial

Puede servirnos incluso para poder desarrollar pruebas y betas de una forma rápida de proyectos más importantes.

La instalación es bien sencilla:

1. Abrimos un navegador.
2. Descargamos el software a partir de la siguiente página https://www.scirra.com/construct2/releases/r252/download
3. Una vez descargado, instalamos el software.

EJEMPLO PRÁCTICO

Otro ejemplo de uso de un motor de videojuego mucho más complejo, pero más polivalente es el uso de libGDX.

Siguiendo la documentación de https://libgdx.com/dev/:

- Podemos integrarlo dentro de un proyecto con Eclipse o Intellij.
- Crear una clase base que cargue un sprite y se mueva por la pantalla.

3.2 Presentación del proyecto

Para comenzar con el proyecto, vamos a seguir la documentación que en la propia página de Scirra aparece como "Beginner's guide to Construct 2" y que iremos describiendo paso a paso y con vídeos de apoyo para que sea mucho más fácil seguir el proceso.

PARA SABER MÁS

Hemos elegido Construct2 debido a la enorme documentación y el foro tan activo existente que siempre es muy importante en el momento del desarrollo:

https://www.scirra.com/tutorials/top

En este punto el primer paso consistirá en abrir un proyecto en blanco tal y como se relata en el siguiente vídeo.

VIDEO DE INTERÉS

En el siguiente vídeo encontrarás una presentación del proyecto:

https://www.youtube.com/watch?v=9rrWspFutTE

3.3 Comenzando por el fondo del proyecto

El primer paso será la incorporación de un fondo a nuestro proyecto. Los recursos que utilizaremos los podemos encontrar en:

- Recursos sobre naves, https://www.gamedev.net/forums/topic/495808-free-airplane-sprite-pack/
- Recursos para el fondo.

Los pasos que debemos seguir son los siguientes:

1. Cambiar los tamaños del proyecto.
2. Seleccionar y renombrar el "Layer" de fondo.
3. Incorporar un nuevo objeto denominado "Tiled Background".
4. Seleccionar el background.
5. Ajustar posición y tamaño.
6. Bloquear el Layer.

VIDEO DE INTERÉS

En el siguiente enlace encontrarás el background del proyecto:

https://www.youtube.com/watch?v=BZvXdV7Fkhk

3.4 Incorporar un jugador

Para incorporar un jugador, seguiremos los pasos que se nos indican a continuación:

1. Crear un nuevo "Layer" y cambiamos el nombre.
2. Incorporar un nuevo objeto denominado "Sprite".
3. Seleccionamos el Sprite.
4. Ajustar posición y tamaño.
5. Cambiar el nombre del Sprite.

VIDEO DE INTERÉS

En el siguiente enlace encontrarás los Sprites del proyecto:

https://www.youtube.com/watch?v=if6VJfPnnJY

3.5 Incorporar comportamientos

A la hora de incluir comportamientos, debemos seguir los siguientes pasos:

1. Seleccionar el jugador.
2. Seleccionar "Behaviors".
3. Añadir "8 directions".
4. Añadir un segundo comportamiento "Bound To Layout".
5. Añadir un tercer comportamiento "Scroll To".

VIDEO DE INTERÉS

En el siguiente vídeo podrás aprender en qué consiste los comportamientos y cómo incluirlos:

https://www.youtube.com/watch?v=P-rELahLBC8

VIDEO DE INTERÉS

En el siguiente vídeo encontrarás la segunda parte sobre los comportamientos:

https://www.youtube.com/watch?v=4tE0BoB6YOI

3.6 Incorporar eventos

Para agregar eventos, es necesario seguir los pasos que nos encontramos a continuación:

1. Crear un nuevo "Sprite". En este caso incorporamos el láser.
2. Añadir al Sprite del láser un "bahaviour" de "Bullet".
3. Cambiar la velocidad.
4. Añadir al proyecto el objeto "Mouse".

5. Añadir los eventos necesarios para que cuando apretemos con el ratón el botón izquierdo dispare el láser.

VIDEO DE INTERÉS

En el siguiente vídeo podrás conocer cómo incorporar eventos:

https://www.youtube.com/watch?v=O0IVaofv7vQ

3.7 Incorporar naves enemigas

La forma de incorporar naves enemigas es siguiendo los siguientes pasos:

1. Incorporar un nuevo objeto denominado "Sprite".
2. Seleccionamos el Sprite que represente a nuestro enemigo.
3. Ajustar posición y tamaño.
4. Cambiar el nombre del Sprite.
5. Duplicaremos los enemigos en el número que queramos.
6. Aplicaremos un "Behaviour" de "Bullet" ajustando la velocidad.
7. Incorporaremos un nuevo evento para que cuando el láser colisione con el enemigo la destruya.

EJEMPLO PRÁCTICO

A partir de lo visto, podríamos hacer más complicadas las naves enemigas. Para ello:

1) Incluimos un nuevo evento de "System every X seconds".
2) Modificamos la velocidad de las naves enemigas aumentándoles su velocidad en 10.

VIDEO DE INTERÉS

En el siguiente vídeo, averiguarás cómo añadir naves enemigas:

https://www.youtube.com/watch?v=KJV0em6gJTk

3.8 Crear jugabilidad en el juego

Para crear jugabilidad en el juego, seguiremos los pasos que se nos indican en los siguientes vídeos viendo cómo incorporar los eventos esenciales para que una nave apunte de manera correcta, para que se produzca una persecución y para añadir sonidos y explosiones.

VIDEO DE INTERÉS

En este vídeo incorporaremos los eventos necesarios para que la nave apunte de una forma mejor:

https://www.youtube.com/watch?v=6426uFS_baE

VIDEO DE INTERÉS

En este vídeo incorporamos los eventos necesarios para que nos persigan:

https://www.youtube.com/watch?v=wqac8-H3vMA

VIDEO DE INTERÉS

En este vídeo incorporamos sonidos y explosiones:

https://www.youtube.com/watch?v=nTZNybi5wO4

3.9 Finalizando el juego

Para finalizar el juego, seguiremos los pasos que se muestran en los siguientes vídeos que engloban cómo añadir una pantalla de inicio, animaciones en el botón y la puntuación.

VIDEO DE INTERÉS

En este vídeo incorporaremos una pantalla de inicio:

https://www.youtube.com/watch?v=c7RcmEdeLNk

VIDEO DE INTERÉS

En este vídeo incorporaremos animaciones en el botón:

https://www.youtube.com/watch?v=_GStFmNdtjQ

VIDEO DE INTERÉS

En este vídeo añadiremos la puntuación:

https://www.youtube.com/watch?v=sPCpk6FXYCY

4. MOTOR DE VIDEOJUEGOS 3D

Aunque ya hemos hecho algún comentario anteriormente sobre los motores 3D, en esta sección nos vamos a centrar en la utilización y las características de uno de ellos. Los motores de juegos 3D son más potentes que los 2D y, en consecuencia, más complicados de aprender.

SABÍAS QUE...

Necesitamos un año de estudio para lograr realizar cosas básicas para juegos en primera persona y varios años para juegos de deportes o de coches.

GameGuru es uno de los motores 3D más fáciles de utilizar debido a que nos ofrece distintas plantillas con las que trabajar. Sin embargo, las características que nos ofrece están muy limitadas.

Cryengine es un motor que nos permite crear escenarios de alta calidad con bastante facilidad. Sin embargo, puede estar muy enfocado para desarrollar un tipo muy concreto de juegos.

Unreal es uno de los motores más usados dentro de la industria. Este motor ofrece una alta calidad de gráficos y muchas funcionalidades. Sin embargo, es un motor bastante difícil de utilizar si no se tiene experiencia.

Unity es otro de los motores más potentes que existen con millones de desarrolladores a nivel mundial. Los recursos y tutoriales son múltiples y muy completos, lo que hace que su popularidad haya crecido. Nos ofrece muchas funcionalidades gratuitas con lo cual tenemos muchas alternativas para desarrollar nuestros juegos. Además, es uno de los motores gráficos más recomendados para la gente que empieza en este mundo. Permite desarrollo de juegos 2D y 3D para múltiples plataformas (móviles, consolas, sistemas operativos, etc.).

Por todo ello, a lo largo de las siguientes secciones, enfocaremos Unity como el motor para desarrollar juegos 3D. Pretendemos realizar una introducción a Unity a través de ejemplos que nos permitirán explorar distintas utilidades básicas y avanzadas de Unity. Sin embargo, hay aspectos que simplemente comentaremos muy brevemente o que quedan más allá del objetivo de esta unidad. Para otro tipo de contenidos extra, se recomienda visualizar la bibliografía, webgrafía y enlaces recomendados.

4.1 Instalación

El primer paso para trabajar con Unity será descargarlo desde la página oficial.

ENLACE DE INTERÉS

Para descargar Unity, accederemos al siguiente link:

https://store.unity.com/es

Como podemos observar, Unity nos ofrece un plan gratuito para principiantes y estudiantes. Esta opción es ideal para iniciarnos con Unity. A pesar de que la instalación es un proceso largo, no tiene mucha complicación. Durante la instalación, deberemos elegir el directorio a instalar, así como los componentes. Por defecto, Unity nos instalará también una versión de Microsoft Visual Studio, que nos permitirá editar código como veremos más adelante.

VIDEO DE INTERÉS

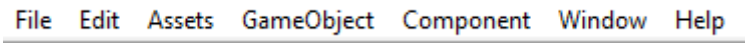
En el siguiente vídeo encontrarás cómo se instala Unity:

https://youtu.be/UkH3s9o_nlE

4.2 Interfaz de Unity

Una vez tenemos Unity instalado, lo ejecutaremos y crearemos un nuevo proyecto tal y como nos sugiere. Una vez dentro de la aplicación, veremos la interfaz principal, la cual podemos dividir en 6 partes diferenciadas. A continuación, describimos las características más importantes de cada una de ellas.

Barra de menú. Esta barra está situada en la parte superior de la ventana. A partir de los distintos menús, podemos acceder a distintas opciones como abrir ficheros, añadir componentes o acceder a otras ventanas.

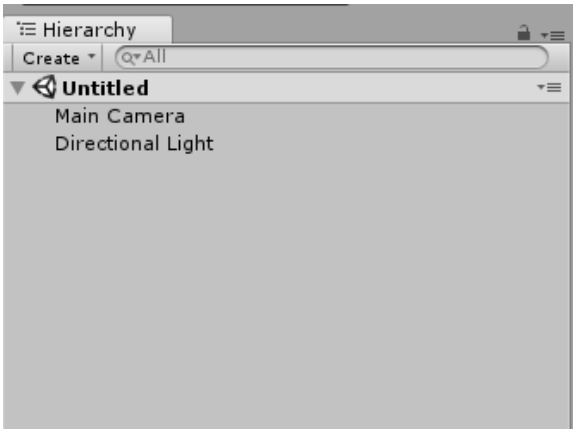


Barra de menú.

Barra de herramientas. Esta barra está situada justo debajo de la barra de menú. En ella, podemos encontrar herramientas para transformar objetos (rotación, escalado, etc.), iniciar la reproducción del juego y pausarla, o gestionar capas.

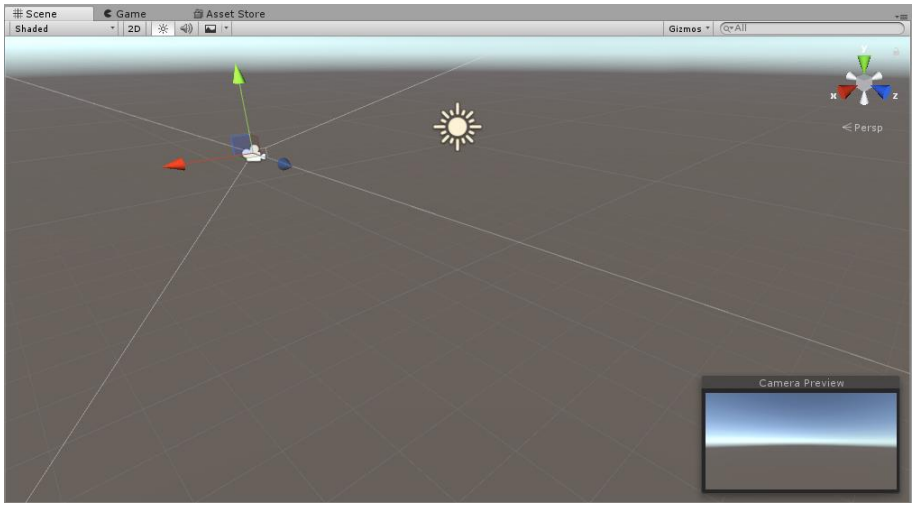
Barra de herramientas.

Ventana de jerarquía. Esta ventana se sitúa a la parte izquierda de la pantalla y debajo de la barra de herramientas. En esta ventana, podemos ver todos los objetos de la escena, crear objetos de distintos tipos, seleccionarlos o borrarlos.



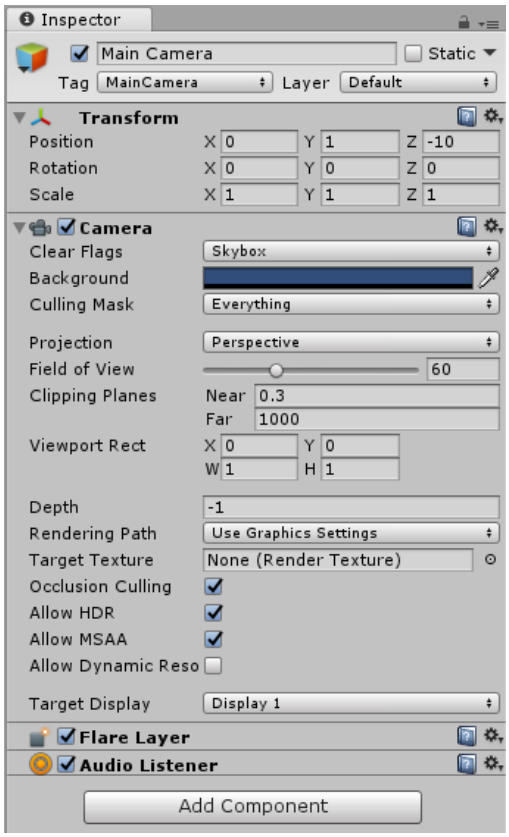
Ventana de jerarquía.

Ventana de la escena. Esta ventana se encuentra en la parte central de la ventana y en ella se concentrará la mayor parte de nuestro trabajo. Aquí crearemos nuestra escena y manipularemos los distintos objetos. También podemos acceder a la vista de la cámara o descargar nuevos recursos para nuestro juego.



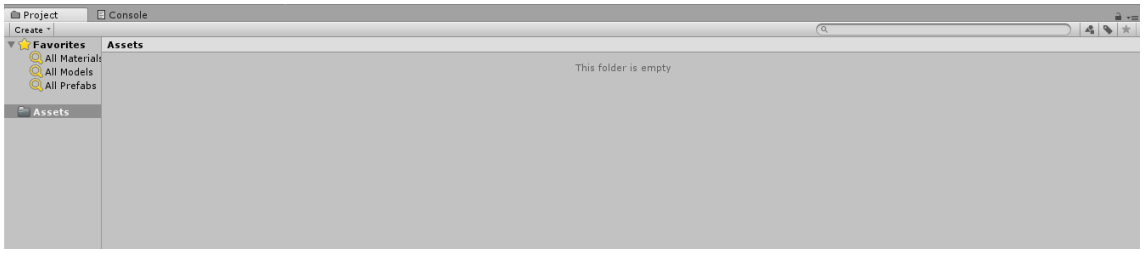
Ventana de la escena.

Ventana de inspector. Esta ventana está a la parte derecha de la pantalla y nos mostrará las propiedades de los distintos objetos que tengamos seleccionados. A través del inspector, podemos realizar acciones como acceder a los componentes del objeto seleccionado (por ejemplo, para transformarlo) o añadir componentes a dicho objeto (por ejemplo, un script).



Ventana de inspector.

Ventana de proyecto. Esta ventana se sitúa en la parte inferior de la pantalla y nos permitirá realizar acciones como crear directorios o gestionar los distintos recursos de nuestro proyecto.



Ventana de proyecto.

PARA SABER MÁS

Si quieres saber más sobre las ventanas de Unity, puedes visitar esta web:

https://docs.unity3d.com/Manual/UsingTheEditor.html

VIDEO DE INTERÉS

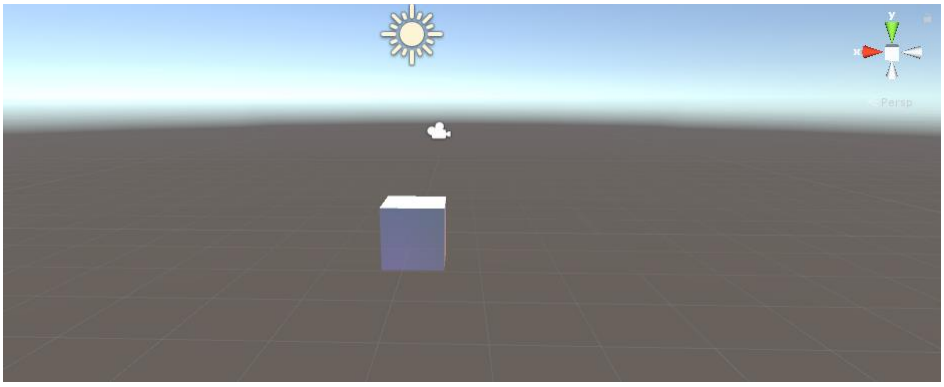
En el siguiente vídeo podemos ver la interfaz de Unity:

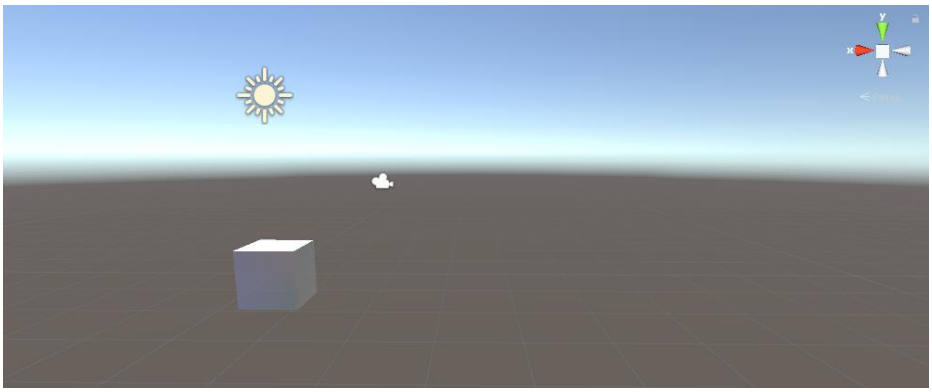
https://youtu.be/34G6F8SFXLg

4.3 Añadiendo objetos a la escena

A medida que vamos añadiendo objetos a nuestra escena, tenemos que gestionarlos en la ventana de la escena. En esta sección, vamos a analizar los controles más habituales que nos permitirán navegar por la escena.

En primer lugar, añadiremos un cubo a la escena, haciendo clic derecho en la ventana de jerarquía y seleccionando 3D object > Cube. Manteniendo la tecla Alt pulsada, veremos que aparece un icono de un ojo y, haciendo clic y moviendo el ratón, podremos orbitar alrededor del espacio de este objeto. Si mantenemos esta tecla pulsada, pero hacemos clic en la rueda central del ratón, podremos ver que podemos mover el plano vertical con el que miramos al objeto. Si mantenemos esta tecla pulsada y hacemos clic derecho, podremos hacer zoom moviendo el ratón hacia arriba y hacia abajo. Esto también lo podemos hacer moviendo la rueda central del ratón.

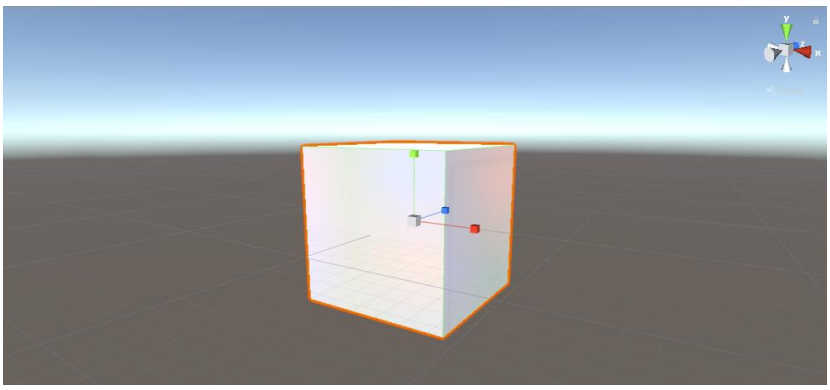




Movimiento de la cámara.

Si hacemos clic en el botón derecho del ratón y sin soltar, podemos movernos hacia distintos puntos de la escena moviendo el ratón. Haciendo clic en el botón derecho y sin soltar, podemos también presionar la tecla "W" para acercarnos al objeto, la tecla "S" para alejarnos, la tecla "A" para movernos hacia la izquierda del objeto y la tecla "D" para movernos a la derecha.

Si haciendo movimientos perdemos de vista nuestro objeto, podemos seleccionarlo en la ventana de jerarquía y presionar la tecla "F" mientras el cursor del ratón está sobre la escena. Esto nos centrará el objeto en la escena.



Objeto centrado.

VIDEO DE INTERÉS

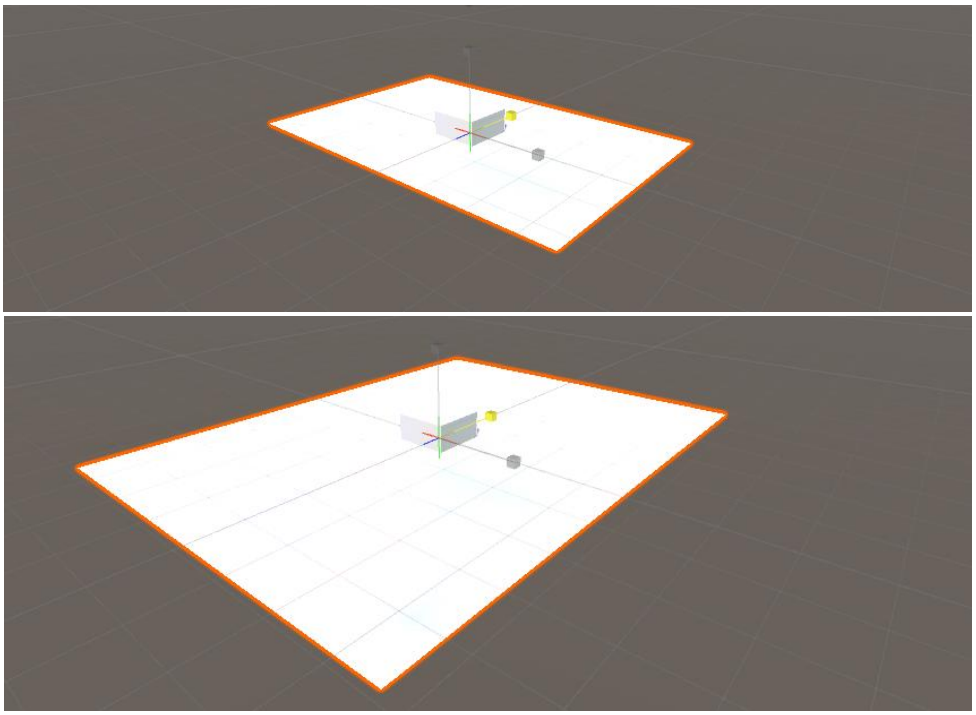
En el siguiente vídeo podemos ver cómo añadir objetos:

https://youtu.be/5aUwfP87EFY

4.4 Transformando objetos

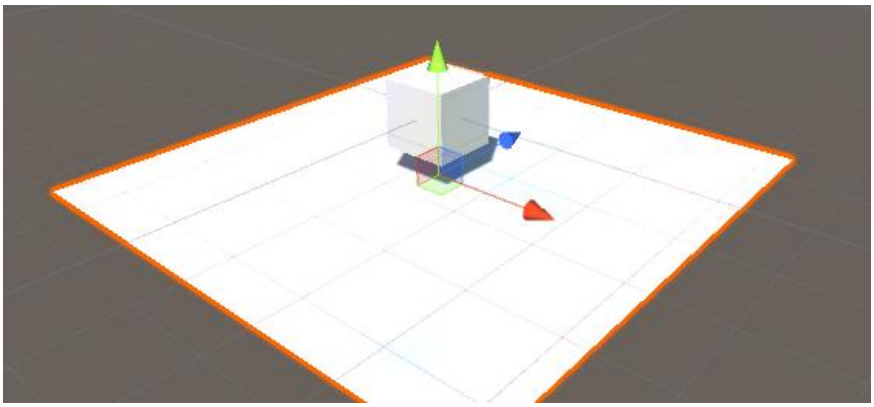
Ya hemos visto cómo añadir objetos a la escena. En esta ocasión, vamos a añadir un plano. Para ello, desde la ventana de jerarquía, haremos clic derecho con el ratón y accederemos a 3D Object > Plane, que nos añadirá un plano en el punto central de nuestra escena.

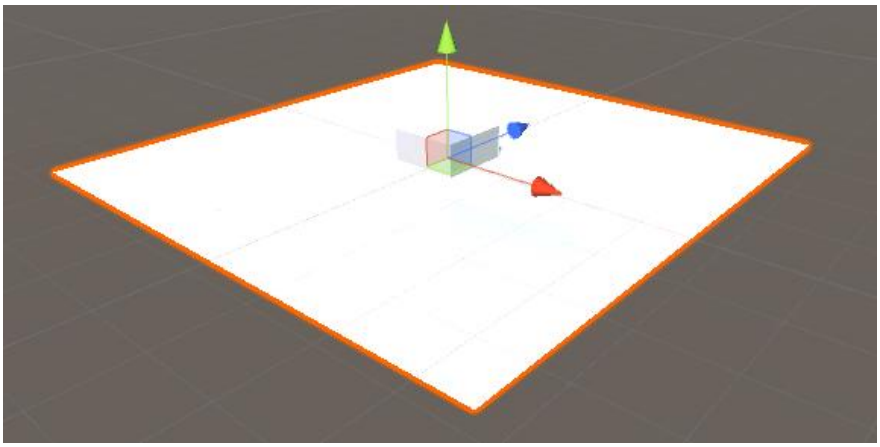
Podemos escalar este plano en cualquiera de los ejes. Para ello, estando pulsado el botón de herramienta de escalado, haremos clic en uno de los ejes y arrastraremos hacia un lado u otro, lo que transformará dicho plano.



Escalado.

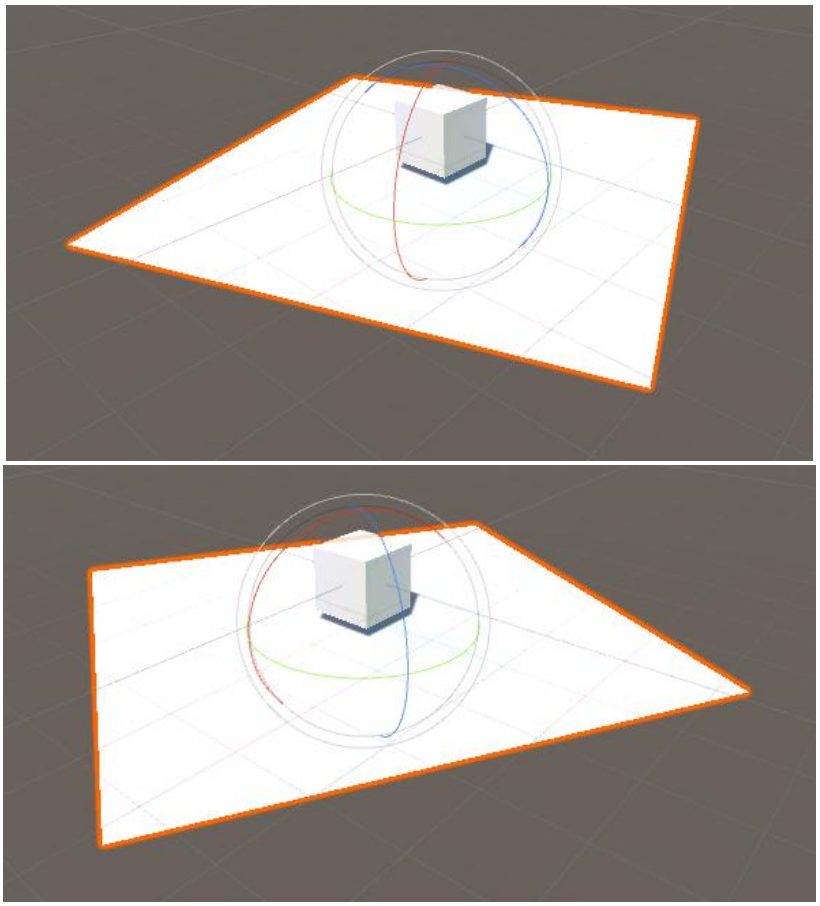
Si hacemos clic en el icono de herramienta de movimiento, podremos mover el plano en cualquiera de los ejes.





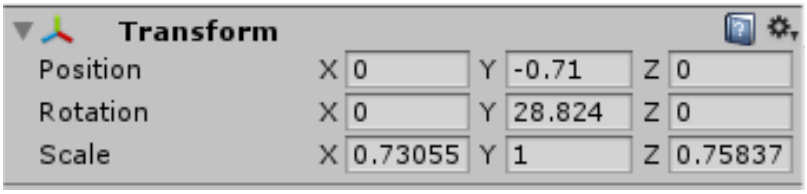
Movimiento.

Con la herramienta de rotación veremos que nos aparece una esfera con unos ejes de colores. Haciendo clic en cualquiera de estos ejes podemos rotar el plano distintos grados en ese eje.



Rotación.

Además de realizar estas transformaciones con las funcionalidades vistas arriba, también es posible cambiar la posición, rotación y escalado a través del componente **Transform** de la ventana de Inspector:



Componente Transform.

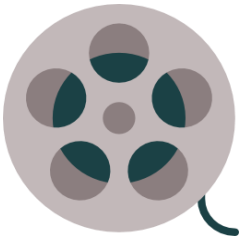
PARA SABER MÁS

Si quieres saber más sobre transformación de objetos, puedes visitar esta web:

https://docs.unity3d.com/es/current/Manual/Transforms.html

COMPRUEBA LO QUE SABES

Una vez has visto cómo añadir y transformar objetos, prueba a añadir una esfera y aplicarle distintos tipos de transformaciones.



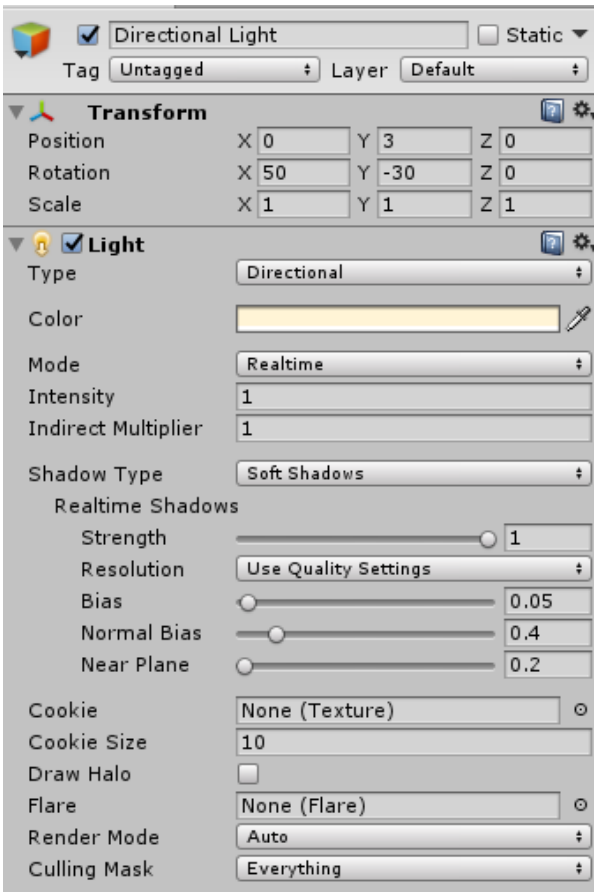
VIDEO DE INTERÉS

En el siguiente vídeo podemos ver cómo transformar objetos:

https://youtu.be/zBCIsLvxlb0

4.5 Iluminación

Hay distintos tipos de luz disponibles en Unity. Cuando creamos un proyecto, por defecto, Unity añade una luz direccional en la escena. Esta luz la podemos encontrar en la ventana de jerarquía y, si hacemos clic, podemos ver las propiedades en la ventana de inspector.



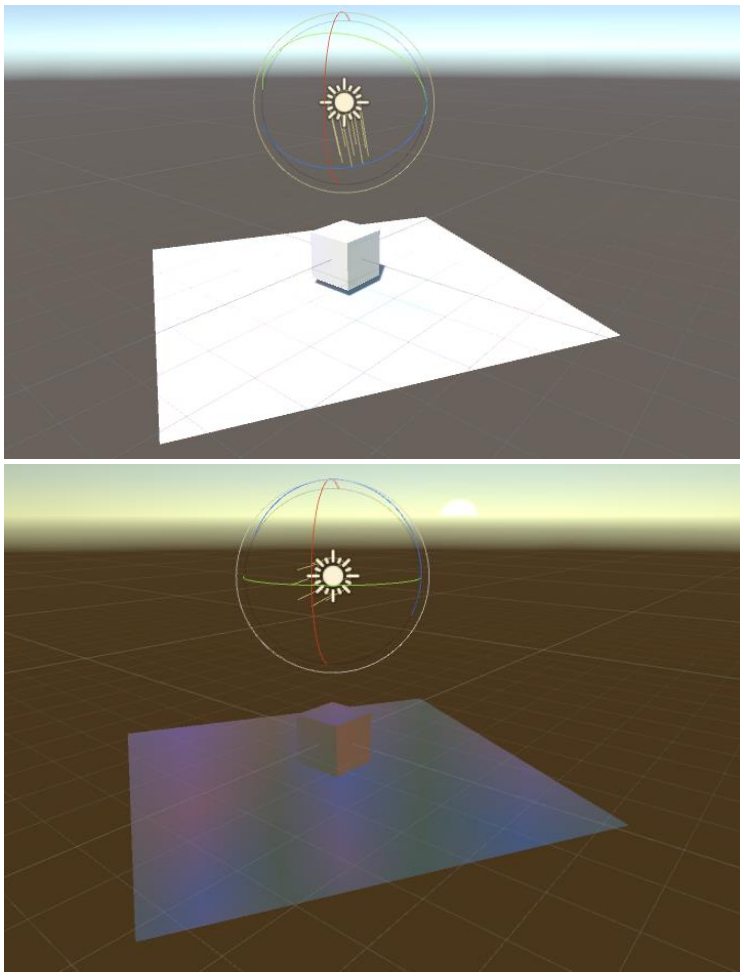
Propiedades de la luz direccional.

En este tipo de luz, el parámetro principal es la dirección de la luz. Para ello, podemos hacer clic en el icono de rotación, el cual nos permitirá cambiar esta dirección.

PARA SABER MÁS

Si quieres saber más sobre las propiedades de iluminación, visita esta web:

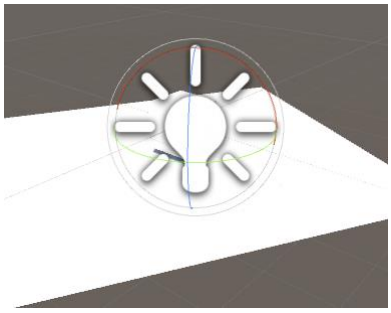
https://docs.unity3d.com/Manual/class-Light.html



Distintos ángulos de luz.

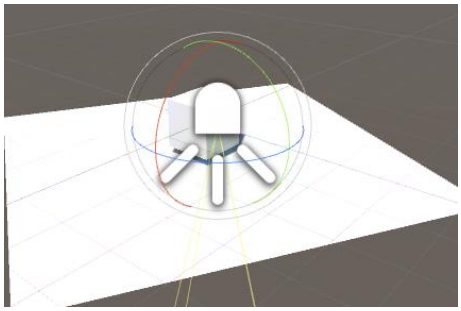
Además de la dirección, podemos cambiar otras propiedades como el color, la intensidad o los tipos de sombras.

Otro tipo de luz que ofrece Unity es un "point light" o luz puntual. Para añadirlo, podemos ir a la ventana de jerarquía y acceder a Create > Light > **Point Light**. Este tipo de luz emite en todas las direcciones. Las propiedades de este tipo de luz son, por ejemplo, el rango que indica la distancia a la que llega la luz o el color.



Luz pointlight.

Otro tipo de luz es la "**spotlight**" o foco que es parecida a la luz direccional, pero con una forma concreta. De esta manera, podemos definir un rango y un ángulo que ajusta la forma del cono de luz. Este tipo de luz es interesante para crear, por ejemplo, un efecto de flash.



Luz spotlight.

ENLACE DE INTERÉS

En el siguiente enlace podrás profundizar sobre los tipos de luz de Unity:

https://docs.unity3d.com/Manual/Lighting.html

VIDEO DE INTERÉS

En el siguiente vídeo podemos ver aspectos de iluminación:

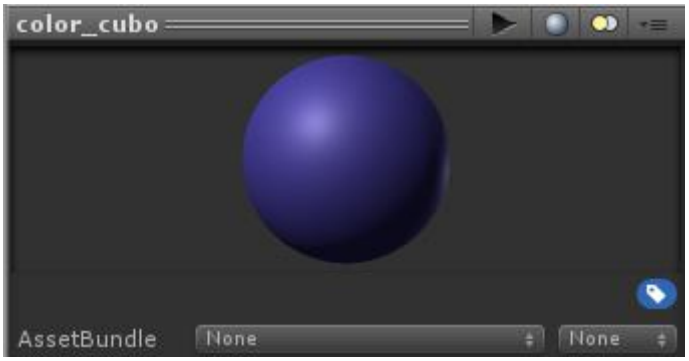
https://youtu.be/_oC_5WLIpo0

4.6 Materiales y texturas

Un material nos va a permitir añadir un color a un objeto. Para ello, podemos importar materiales o crear un nuevo con la opción Create > Material desde la ventana del proyecto. Si tenemos varios materiales, sería interesante crear un directorio y almacenarlos todos ahí.

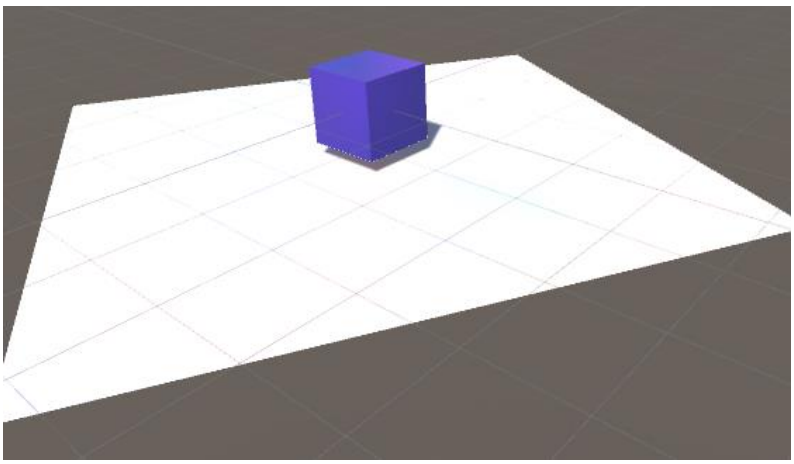
Materiales.

La primera propiedad que podemos ver en un material es "albedo". Esta propiedad representa el color de un objeto en su forma pura sin considerar información de iluminación. Si hacemos clic en esta propiedad, se nos abrirá una paleta en donde podemos elegir un color para este material. Una vez seleccionamos el color, podemos previsualizar el efecto en la parte inferior de la ventana de inspector.



Previsualización de un material.

Para aplicar este material a un objeto de la escena, podemos seleccionar el material desde la ventana de proyecto y arrastrarlo al objeto. En la imagen podemos ver cómo el color del cubo ha cambiado al material definido.



Material añadido a un objeto.

Un renderizado estándar (propiedad "Shader") es un método de texturización que nos permite crear una apariencia consistente a través de nuestros materiales. Cabe observar que una de las opciones del material nos indica el "Rendering mode", en donde podemos elegir varios modos de renderización. Por defecto, el que nos aparece es "Opaque", pero podríamos probar otros tipos. Como podemos ver, podemos crear materiales con mucha más complejidad dependiendo de las necesidades de nuestro juego.

Material aplicado a un objeto.

Una vez hemos añadido el material a nuestro objeto, podemos ver que, en las propiedades de este objeto, aparece el material aplicado. En esta imagen se puede ver que el material ha sido asignado al render de mallas, que se encarga de renderizar cualquier malla tridimensional.

PARA SABER MÁS

Si quieres saber más sobre las propiedades de material, puedes visitar esta web:

https://docs.unity3d.com/es/current/Manual/class-Material.html

ENLACE DE INTERÉS

En el siguiente enlace encontrarás más información acerca de la creación y uso de materiales:

https://docs.unity3d.com/Manual/Materials.html

VIDEO DE INTERÉS

En el siguiente vídeo podemos ver cómo usar materiales:

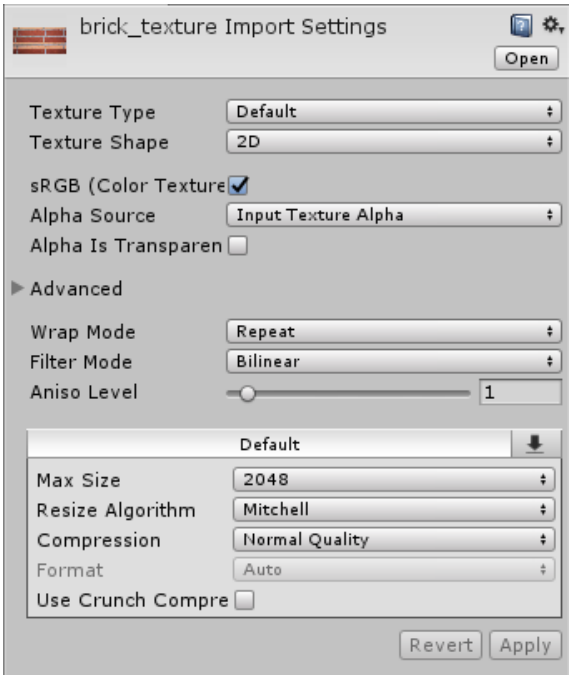
https://youtu.be/FuF0nUd3I9I

A parte de la funcionalidad vista arriba, Unity también nos ofrece la posibilidad de usar texturas sobre los objetos. Una textura es un fichero de imagen que podemos aplicar a un material para darle a una malla una superficie con la textura seleccionada.

SABÍAS QUE…

Aunque podemos usar como textura cualquier fichero de imagen soportado por Unity, lo más habitual será crear una textura en un programa de edición de imágenes como Photoshop o Gimp.

A modo de ejemplo, vamos a copiar el fichero "brick_texture.jpg" que podemos encontrar en la carpeta de "Recursos" en una carpeta nueva dentro de "Assets" llamada "Texturas". En este momento, veremos que la textura nos aparece en la ventana del proyecto y si hacemos clic en ella, podemos ver sus propiedades.



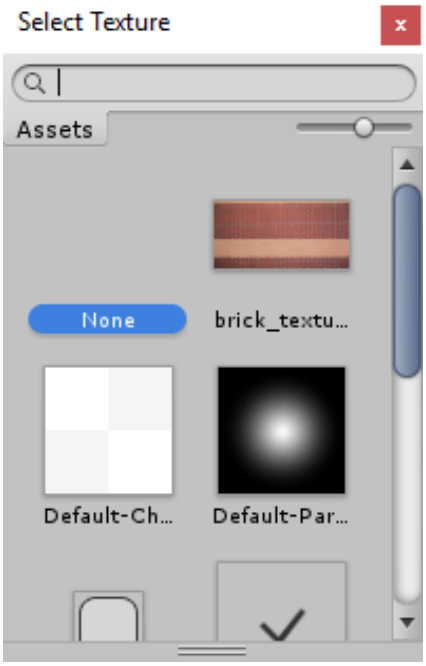
Propiedades de la textura.

ENLACE DE INTERÉS

Si quieres descargar texturas gratuitas, puedes visitar esta web:

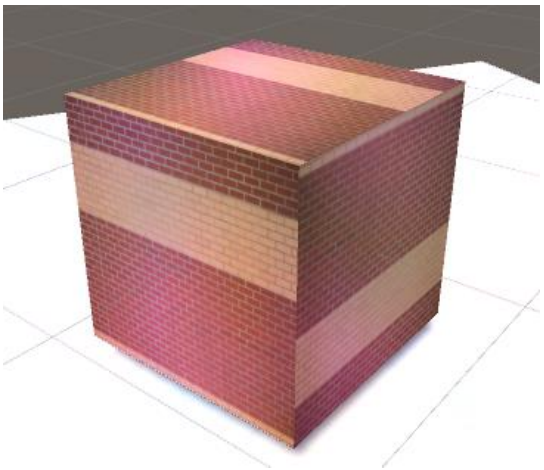
http://www.texturex.com/

Una vez tengamos la textura, la podemos aplicar a cualquier objeto de la misma forma que hemos hecho anteriormente. Para ello, crearemos un nuevo material, pero esta vez, en vez de seleccionar un color de la propiedad albedo a través de la paleta, haremos clic en el pequeño círculo que hay justo a la izquierda de esta propiedad. Esto nos abrirá una pequeña ventana de texturas, incluyendo la nueva que hemos añadido.



Selección de textura.

La seleccionaremos y habremos creado un nuevo material con esta textura. Para aplicar el material a la textura, bastará con repetir el proceso que hemos realizado anteriormente, con lo que nuestro cubo, ahora quedará con la textura.

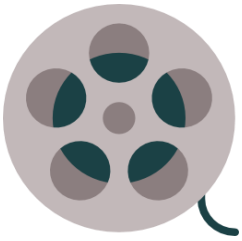


Textura aplicada a un objeto.

PARA SABER MÁS

En el siguiente enlace podrás ampliar tus conocimientos sobre las texturas:

https://docs.unity3d.com/Manual/Textures.html



VIDEO DE INTERÉS

En el siguiente vídeo podemos ver cómo usar texturas:

https://youtu.be/U3_rmT9R61A

4.7 Scripts, lenguajes y API gráfico

Cuando creamos juegos, en algún momento necesitaremos definir la interacción del jugador, de manera que se produzcan distintos comportamientos. Para ello, utilizaremos scripts con código.

En Unity podemos crear scripts con C# o con JavaScript. En general, podemos encontrar mayores referencias sobre código C#, por lo que, en esta unidad, usaremos este lenguaje. Para crear un script, utilizaremos el mismo proceso que anteriormente: Create > C# script. Esto creará un fichero con código predefinido que podemos previsualizar en la ventana de inspector.

RECUERDA

Para organizar mejor los distintos recursos, sería mejor crear un directorio para cada tipo de recurso, en este caso, para scripts.

Assembly Information

Filename Assembly-CSharp.dll

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class NewBehaviourScript : MonoBehaviour {

        // Use this for initialization
        void Start () {

        }

        // Update is called once per frame
        void Update () {

        }
}
```

Código del script.

Para editar este código, bastará hacer doble clic en este script, lo nos abrirá Visual Studio.

El código que nos ha creado representa una clase para definir el comportamiento que queremos que tenga el objeto. En este código podemos ver dos métodos:

- **Start**: Se ejecuta cuando empieza el juego.
- **Update**: Se ejecuta cada frame mientras se ejecuta el juego.

A modo de ejemplo, vamos a hacer un script que rote nuestro objeto en un determinado eje. Para ello, añadiremos el siguiente código al método Update:

```
void Update () {
    transform.Rotate(Vector3.up);
}
```

ENLACE DE INTERÉS

En los siguientes enlaces encontrarás toda la documentación sobre la clase Transform y el método Rotate.

Clase Transform:
https://docs.unity3d.com/ScriptReference/Transform.html

Método Rotate:
https://docs.unity3d.com/ScriptReference/Transform.Rotate.html

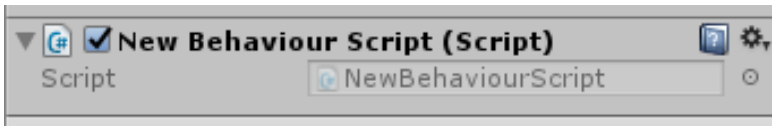
PARA SABER MÁS

En el siguiente enlace podrás encontrar toda la documentación sobre la clase Vector3:

https://docs.unity3d.com/ScriptReference/Vector3.html

Cada objeto de la escena tiene una propiedad Transform que guarda la posición, rotación y escala del objeto. Para manipular estos valores, bastará con cambiar estos valores de una forma similar a como se ve en el código anterior. Vector3 es una estructura para almacenar posiciones en 3D y direcciones. Con la propiedad "up", estamos haciendo referencia al Vector3(0,1,0).

Una vez tenemos este script, podemos hacer clic y arrastrarlo al objeto al que deseemos aplicárselo, y lo podremos ver como un componente en el panel Inspector del objeto.



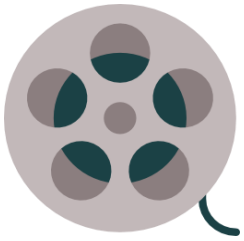
Componente script.

Para ejecutar el juego, tendremos que dar al botón de "play" que hay en la barra de herramientas.

Ejecución del juego.

COMPRUEBA LO QUE SABES

Una vez has visto cómo crear scripts, prueba a rotar un cubo en los tres ejes posibles.



VIDEO DE INTERÉS

En el siguiente vídeo podemos ver cómo escribir y ejecutar scripts:

https://youtu.be/Bf6lKbkCjus

RESUMEN FINAL

En esta unidad, hemos visto conceptos para el desarrollo de videojuegos en 2D y 3D.

En primer lugar, hemos visto conceptos generales e históricos del desarrollo de videojuegos. Después, nos hemos centrado en la parte del diseño de videojuegos y motores de videojuegos. Finalmente, hemos aplicado conceptos totalmente prácticos sobre dos motores de videojuegos muy comunes para el desarrollo de videojuegos 2D y 3D: Construct 2 y Unity, respectivamente.